

Unpublished Post. This URL is public but secret, so share at your own discretion. [Click to edit](#)



jmake

Edit Stats

Integrating NVIDIA PhysX 5.6 with Unity engine: A Journey into Deformable GPU Physics

#unity3d #cpp #csharp #gamedev

[Watch the video on YouTube](#)

Integrating a native C++ physics engine with a C# game engine isn't for the faint of heart — especially when it involves GPU-accelerated deformable bodies. But that's exactly what I tackled by bringing **NVIDIA PhysX 5.6** into **Unity** using **SWIG** on **Windows 11**.



Why PhysX 5.6?

While Unity offers a solid built-in physics engine, it's limited when it comes to complex **deformable bodies** — particularly those optimized for GPU execution. PhysX 5.6 introduced a robust framework for GPU-based simulations, including soft and deformable body dynamics, ideal for high-performance scenarios like:

- Real-time VFX
- Soft-body destruction
- Scientific simulation
- Next-gen gameplay mechanics



Bridging C++ and C# with SWIG

To make PhysX 5.6 accessible from Unity's C# environment, I used **SWIG (Simplified Wrapper and Interface Generator)** to generate C# bindings for the C++ API.

Steps:

1. Created SWIG .i interface files for required PhysX modules.
2. Configured SWIG to target C# with platform-specific bindings.
3. Built a native C++ DLL exposing necessary APIs.
4. Wrapped everything into a managed .NET layer callable from Unity.

Result: Unity scripts could call PhysX GPU deformable body functions directly, with zero core reimplementation.

Focus: Deformable Bodies on GPU

PhysX 5.6's GPU deformable body system delivers:

-  **Real-time deformation**
-  **High parallelism** via CUDA
-  **Material-aware soft-body simulation**

It's perfect for use cases like **soft tissue, rubber, clay**, and **cloth-like physics** — all at **real-time performance**.

Challenges Faced

- SWIG limitations when wrapping template-heavy PhysX types
- C++ to C# marshalling edge cases (especially with pointers and memory alignment)
- Unity/Mono native interop quirks
- Debugging native crashes inside the managed runtime 🤖

Results

The integration allowed Unity to simulate GPU-deformable physics beyond its built-in capabilities, with a clean and usable C# API. It's a viable approach for:

- Researchers
- Simulation developers
- VFX-heavy game projects
- Anyone pushing Unity beyond default physics limits

Implementation Details

Integrating PhysX 5.6 with Unity wasn't just about linking libraries — it required careful design across native and managed boundaries. In this section, I'll break down the key architectural decisions, how SWIG was configured, and what it took to get deformable GPU physics working reliably inside Unity's C# runtime on Windows 11.

PhysX 5.6 Compilation: Local and CI Setup

Getting PhysX 5.6 to build on Windows 11 required automating both **local compilation** and **CI builds via GitHub Actions**. For this, I created a PowerShell script to handle the environment setup, Visual Studio detection, and Ninja-based builds.

The core idea: wrap the official PhysX project generation flow in reusable functions that prepare and compile CPU- or GPU-targeted builds with minimal manual intervention.

Key tasks handled in this script include:

- Automatically locating and loading the correct **Visual Studio developer environment** using `vswhere`
- Configuring CMake presets for **Ninja** or **MSBuild** generators
- Switching between **CPU-only** and **GPU (CUDA)** PhysX configurations
- Generating project files via `generate_projects.bat`
- Building via Ninja and exporting PhysX as a `.zip` for deployment or use in Unity

This build script allowed consistent, reproducible compilation on both local machines and CI runners.

PowerShell Build Script (Simplified)

```
function CL_SETUP {
    $VSWHERE="C:\ProgramData\Chocolatey\bin\vswhere.exe"
    $VSTOOLS = &($VSWHERE) -latest -products * -requires Microsoft.VisualStudio.Component

    if ($VSTOOLS) {
        $VSTOOLS = Join-Path $VSTOOLS 'Common7\Tools\vsdevcmd.bat'
        if (Test-Path $VSTOOLS) {
            cmd /s /c " ""$VSTOOLS"" -arch=x64 -host_arch=x64 $args && set" |

```

```

Where { $_ -match '(\w+)=(.*)' } |
ForEach { $null = New-Item -Force -Path "Env:\$($Matches[1])" -Value $Matches[2]
}

}

cl.exe
cmake.exe --version
ninja.exe --version
}

function CompilePhysX {
    Set-Location physx
    .\generate_projects.bat vc16win64
    Set-Location compiler\vc16win64
    ninja.exe -f build-release.ninja -j4 install sdk_gpu_source_bin\all
    Set-Location ..
    Compress-Archive -Path "physx\PhysX50" -DestinationPath "PhysX50.zip"
}

$EXECUTION_PATH = Get-Location
try { cl.exe } catch { CL_SETUP }
CompilePhysX

```

💡 You can adjust the .xml preset files to switch between CPU and GPU builds, update compiler versions (vc16 → vc17), or change the generator (e.g., ninja vs. Visual Studio).

CMake Setup with CUDA, PhysX & SWIG

To integrate PhysX with Unity via a native C++ bridge, I used a CMake-based build system that handles:

- Static linking of PhysX 5.6
- CUDA support for GPU-based deformables
- SWIG to generate C# bindings callable from Unity

The structure looked like this:

```

/MyProject/
├─ CMakeLists.txt
├─ configuration.i # SWIG interface file

```

```

└─ spicytech.cpp # C++ glue layer
└─ Unity/ # Sample Unity C# test
└─ Assets/Plugins/ # Output destination for Unity
└─ external/PhysX50/ # PhysX SDK

```

Here's the CMake configuration that made it all work:

```

cmake_minimum_required(VERSION 3.16)
cmake_policy(SET CMP0122 NEW) # Ensures .NET-style naming in SWIG-generated C#

project(spicytech CXX C CUDA)

set(CMAKE_CXX_STANDARD 11)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_BUILD_TYPE Release)
set(CMAKE_MSVC_RUNTIME_LIBRARY "MultiThreaded$<$<CONFIG:Debug>:Debug>")

# CUDA
find_package(CUDAToolkit REQUIRED)
get_target_property(cuda_runtime_location CUDA::cudart LOCATION)

# PhysX
set(PHYSX_LIB_DIR "${PHYSX_ROOT_DIR}/bin/win.x86_64.vc143.mt/release")
file(GLOB PHYSX_STATIC_LIBS "${PHYSX_LIB_DIR}/*_64.lib")
include_directories("${PHYSX_ROOT_DIR}/include")

# Sources
file(GLOB SRCS "${CMAKE_CURRENT_SOURCE_DIR}/*.cpp")
add_library(LibrarySnippet STATIC ${SRCS})

# SWIG
find_package(SWIG REQUIRED)
include(${SWIG_USE_FILE})
set(CMAKE_SWIG_OUTDIR ${CMAKE_BINARY_DIR}/Release)
set_source_files_properties(configuration.i PROPERTIES CPLUSPLUS ON)

set(MODULE_NAME "ModuleName")
swig_add_library(LibraryName
    LANGUAGE csharp
    SOURCES configuration.i spicytech.cpp
)

swig_link_libraries(LibraryName ${PHYSX_STATIC_LIBS} LibrarySnippet)
target_include_directories(${SWIG_MODULE_LibraryName_REAL_NAME} PRIVATE ${CUDAToolkit_]

```

```
target_link_libraries(${SWIG_MODULE_LibraryName_REAL_NAME} CUDA::cudart)

# Unity integration: move outputs post-build
add_custom_command(TARGET LibraryName POST_BUILD
  COMMAND ${CMAKE_COMMAND} -E make_directory "${CMAKE_SOURCE_DIR}/Assets/Plugins"
  COMMAND ${CMAKE_COMMAND} -E make_directory "${CMAKE_SOURCE_DIR}/Assets/Plugins/SpicyTech"

  COMMAND ${CMAKE_COMMAND} -E rename
    "$<TARGET_FILE:LibraryName>"
    "${CMAKE_SOURCE_DIR}/Assets/Plugins/LibraryName.dll"

  COMMAND ${CMAKE_COMMAND} -E copy_directory
    "${CMAKE_SOURCE_DIR}/Unity"
    "${CMAKE_SOURCE_DIR}/Assets/Plugins/Scripts"

  COMMAND ${CMAKE_COMMAND} -E copy_directory
    "${CMAKE_SWIG_OUTDIR}"
    "${CMAKE_SOURCE_DIR}/Assets/Plugins/SpicyTech"
)
```

This setup builds a static native C++ library, wraps it with C# bindings using SWIG, and organizes output files for Unity's Assets/Plugins structure — ready for direct use inside the Unity Editor.

Wrapping C++ with SWIG

With CMake generating both the C++ static library and SWIG C# bindings, the next step was defining what parts of the native API Unity (C#) should access.

configuration.i: SWIG Interface File

```
%module ModuleName

%{
#include "spicytech.hpp"
%}

#include "std_vector.i"
%template(FloatVector) std::vector<float>;

#include "typemaps.i"
```

```
%include "arrays_csharp.i"

%apply int INOUT[] { int* outArray2 };
%apply float INOUT[] { float* outArray1 };

%apply int* OUTPUT { int* rows, int* cols };

#include "spicytech.hpp"
```

This interface file:

Exposes the SpicyX class and other native functions to C#

Uses SWIG typemaps to marshal float* and int* arrays across the C++/C# boundary

Allows automatic C# array interop for mesh data and deformable simulation outputs



spicytech.hpp: Native Interface Layer

```
#pragma once
#include <vector>
#include <iostream>
#include <algorithm>

int CudaVersion();
float PhysxVersion();
int InitPhysics();
void CleanupPhysics();
int StepPhysics(int, std::vector<std::vector<int>>&, std::vector<std::vector<float>>&);
void UnityInit(float* outArray1, int n1, int* outArray2, int n2);

class SpicyX {
public:
    SpicyX() {}

    int Init() {
        nDeformables = InitPhysics();
        return nDeformables;
    }

    void MeshAdd(float* outArray1, int n1, int* outArray2, int n2) {
        UnityInit(outArray1, n1, outArray2, n2);
    }
}
```

```

    }

    int Step(int i) {
        return StepPhysics(i, Triangles, PositionsInvMass);
    }

    int nBodies() { return PositionsInvMass.size(); }

    int PositionsInvMassSize(int i) { return PositionsInvMass[i].size(); }

    void PositionsInvMassGet(int i, float* outArray1) {
        std::copy(PositionsInvMass[i].begin(), PositionsInvMass[i].end(), outArray1);
    }

    int TrianglesSize(int i) { return Triangles[i].size(); }

    void TrianglesGet(int i, int* outArray2) {
        std::copy(Triangles[i].begin(), Triangles[i].end(), outArray2);
    }

    void Finish() { CleanupPhysics(interactive); }

private:
    std::vector<std::vector<int>> Triangles;
    std::vector<std::vector<float>> PositionsInvMass;
    int nDeformables;
    bool interactive;
};

```

This C++ class acts as a façade for PhysX interaction:

Provides methods that Unity can call (Init, Step, MeshAdd, Finish, etc.)

Buffers and returns deformable body vertex and triangle data



test.cs: A Minimal C# Consumer

Here's a snippet from a basic .NET test that calls the generated C# bindings. The example creates a cube composed of 8 vertices and 12 triangles. It is then simulated as a GPU-based deformable object using the SpicyX wrapper for PhysX.

```

using System;
using System.Runtime.InteropServices;

class test {
    [DllImport("kernel32.dll")]
    static extern bool SetDllDirectory(string path);

    static void Main() {
        SetDllDirectory(".");

        Console.WriteLine("CUDA Version: " + ModuleName.CudaVersion());
        Console.WriteLine("PhysX Version: " + ModuleName.PhyxVersion());

        using (SpicyX sim = new SpicyX()) {
            MeshCreateCube(sim);
            int count = sim.Init();
            Console.WriteLine("nDeformables: " + count);

            for (int i = 0; i < 10; i++) {
                sim.Step(i);
                float[] vertices = new float[sim.PositionsInvMassSize(0)];
                int[] triangles = new int[sim.TrianglesSize(0)];

                sim.PositionsInvMassGet(0, vertices);
                sim.TrianglesGet(0, triangles);

                Console.WriteLine("vertex sample: " + string.Join(", ", vertices[..3]));
                Console.WriteLine("triangle sample: " + string.Join(", ", triangles[..3]));
            }

            sim.Finish();
        }
    }

    static void MeshCreateCube(SpicyX obj) {
        float[] vertices = new float[] {
            0.5f, -0.5f, -0.5f, 0.5f, -0.5f, 0.5f, -0.5f, -0.5f, 0.5f, -0.5f, -0.5f, -0.5f,
            0.5f, 0.5f, -0.5f, 0.5f, 0.5f, 0.5f, -0.5f, 0.5f, 0.5f, -0.5f, 0.5f, -0.5f
        };

        int[] triangles = new int[] {
            1,2,3, 7,6,5, 4,5,1, 5,6,2, 2,6,7, 0,3,7,
            0,1,3, 4,7,5, 0,4,1, 1,5,2, 3,2,7, 4,0,7
        };
    }
}

```

```
obj.MeshAdd(vertices, vertices.Length, triangles, triangles.Length);  
}  
}
```

Unlike rigid bodies, the vertex positions from `PositionsInvMassGet` change each frame due to deformation. This per-frame data can be sent to Unity (via C#) for real-time visualization of the deforming cube.

The mesh remains connected the same way (same triangle indices), but the shape morphs based on GPU physics simulation.

This is a minimal but complete example of how to run a deformable simulation and fetch results frame-by-frame for real-time rendering.

How It Works

- **MeshAdd:** This method loads your mesh data—vertices and triangles—into the simulation. Think of it as telling the physics engine what shape you want to simulate as a deformable object.
- **Init:** Prepares and starts the simulation environment. You need to call this before stepping through the simulation.
- **Step:** Advances the simulation by one frame or iteration. This updates the physics calculations on the GPU.
- **PositionsInvMassGet & TrianglesGet:** These methods fetch the current simulation results.
 - **PositionsInvMassGet** returns the updated vertex positions (with inverse mass), which change every frame because the object deforms.
 - **TrianglesGet** returns the mesh connectivity (triangles), which stays the same throughout the simulation.

This approach lets you simulate soft bodies on the GPU with PhysX and easily pull updated vertex data into Unity or any C# application for real-time rendering of deformable objects.

Unity Script

Next Unity MonoBehaviour script demonstrates how to integrate and visualize deformable and rigid body physics using the SpicyX simulation wrapper.

```
using UnityEngine;
using System.Collections.Generic;

public class SpicyXPlugging1 : MonoBehaviour
{
    public GameObject[] softObjects;

    SpicyX sx;
    List<VertexVisualizer> visualizersDeformables = new List<VertexVisualizer>();

    delegate void MeshAddDelegate(
        float[] vertices,
        int verticesLength,
        int[] triangles,
        int trianglesLength,
        bool kinematic
    );

    void OnDisable()
    {
        VisualizersClear();
    }

    void Start()
    {
        VisualizersStart();
    }

    void FixedUpdate()
    {
        VisualizersEvolve();
    }

    void VisualizersStart()
    {
        if (sx != null) return;

        sx = new SpicyX();
        DeformablesInit(sx);
        sx.Init();
        VisualizersDeformablesCreate();
    }
}
```

```

    }

    void DeformablesInit()
    {
        foreach (GameObject softObject in softObjects)
        {
            MeshCreate(sx, softObject, new MeshAddDelegate(sx.SoftAdd));
        }
    }

    void VisualizersDeformablesCreate()
    {
        foreach (GameObject softObject in softObjects)
        {
            GameObject go = new GameObject("Soft_" + visualizersDeformables.Count);
            VertexVisualizer visualizer = go.AddComponent<VertexVisualizer>();
            visualizersDeformables.Add(visualizer);
        }
    }

    void VisualizersEvolve()
    {
        sx.Step();

        int nBodies = sx.nBodies();
        for (int iBody = 0; iBody < nBodies; iBody++)
        {
            int nbVertices = sx.PositionsInvMassSize(iBody);
            int nTriangles = sx.TrianglesSize(iBody);

            float[] vertices = new float[nbVertices];
            sx.PositionsInvMassGet(iBody, vertices);

            VertexVisualizer visualizer = visualizersDeformables[iBody];

            if (visualizer.vertices == null || visualizer.vertices.Length != vertices.Length)
                visualizer.vertices = new float[nbVertices];
            System.Array.Copy(vertices, visualizer.vertices, vertices.Length);

            int[] triangles = new int[nTriangles];
            sx.TrianglesGet(iBody, triangles);

            if (visualizer.triangles == null || visualizer.triangles.Length != triangles.Length)
                visualizer.triangles = new int[nTriangles];
            System.Array.Copy(triangles, visualizer.triangles, triangles.Length);
        }
    }

```

```
}  
}  
  
void MeshCreate(  
    GameObject obj,  
    MeshAddDelegate function  
)  
{  
    Mesh mesh = obj.GetComponent<MeshFilter>().mesh;  
  
    int[] triangles = mesh.triangles;  
    int nTriangles = triangles.Length / 3;  
  
    Vector3[] localVertices = mesh.vertices;  
    int nVertices = localVertices.Length;  
  
    Matrix4x4 localToWorld = obj.transform.localToWorldMatrix;  
    Vector3[] worldVertices = new Vector3[localVertices.Length];  
    for (int i = 0; i < localVertices.Length; i++)  
        worldVertices[i] = localToWorld.MultiplyPoint3x4(localVertices[i]);  
  
    float[] vertices = VerticesToArray(worldVertices);  
  
    function(vertices, vertices.Length, triangles, triangles.Length, false);  
}  
  
float[] VerticesToArray(Vector3[] meshVertices)  
{  
    float[] array = new float[meshVertices.Length * 3];  
    for (int i = 0; i < meshVertices.Length; i++)  
    {  
        array[i * 3 + 0] = meshVertices[i].x;  
        array[i * 3 + 1] = meshVertices[i].y;  
        array[i * 3 + 2] = meshVertices[i].z;  
    }  
    return array;  
}  
  
void VisualizersClear()  
{  
    foreach (var visualizer in visualizersDeformables)  
    {  
        if (visualizer != null)  
            Destroy(visualizer.gameObject);  
    }  
}
```

```

    }
    visualizersDeformables.Clear();

    if (sx != null)
    {
        sx.Finish();
        sx = null;
    }
}
}

```

Core Workflow

This script drives a deformable simulation using a custom physics engine (`spicyX`). It loads mesh data from Unity `GameObject` s, initializes deformable bodies, and updates visuals per physics step.

Initialization

- `Start()` → Calls `VisualizersStart()`:
 - Instantiates the `spicyX` simulation engine.
 - Calls `DeformablesInit()` to extract mesh data from each object in `softObjects`.
 - Sends vertex/triangle data to `spicyX` using `SoftAdd` via a delegate.
 - Creates one `VertexVisualizer` `GameObject` per soft object to render its deformation.

Simulation Update (`FixedUpdate()`)

- Each frame:
 - `sx.Step()` advances the physics simulation.
 - Updated vertex and triangle data is fetched per body:
 - `PositionsInvMassGet()` → Vertex positions.
 - `TrianglesGet()` → Triangle indices.
 - The corresponding `VertexVisualizer` is updated with the new mesh data.

Mesh Handling (`MeshCreate()`)

- Reads the `Mesh` component from each `softObject`.
- Transforms local vertices to world space.
- Converts `Vector3[]` vertices to a `float[]` array for compatibility.

- Sends this data to the simulation via a `MeshAddDelegate`.

Cleanup (`OnDisable()` / `VisualizersClear()`)

- Destroys all visualizer `GameObjects`.
- Calls `sx.Finish()` to release simulation memory.

Top comments (0) ⚡

[Code of Conduct](#) • [Report abuse](#)



jmake

JOINED

Apr 25, 2025

Trending on DEV Community 🔥



I'm 16, almost finished GCSEs — this summer, I'm trying to earn my first £1k online

[#productivity](#) [#learning](#) [#webdev](#) [#programming](#)



The Story Behind MultiDrive

[#programming](#) [#csharp](#) [#dotnet](#) [#discuss](#)



So You Want to Be a Senior Engineer...

#discuss #programming #beginners #productivity
